ことばの地図

埋め込み — 文字の意味が「場所」になる

rkskmt

前回の振り返り

- one-hot × 行列 W → softmax で、バイグラム表を勾配降下法で学習できた

> **クイズ**
>
> one-hot ベクトル（1が1個、あとは0）と行列の掛け算。 計算機は75×75＝5,625回の掛け算を**本当に**する必要がある？

- するわけがない。0を掛けるところは全部消える — 残るのは……

種明かし：行を1本選んでいるだけ

Code

```
import torch
import torch.nn.functional as F

W = torch.arange(12.0).reshape(4, 3)       # 小さな行列で実験（語彙4、行の長さ3）
x = torch.tensor([2])                      #  「2番の文字」
X = F.one_hot(x, num_classes=4).float()

print(W)
print(X @ W)           # one-hot × 行列
print(W[2])            # 行列の2行目
```

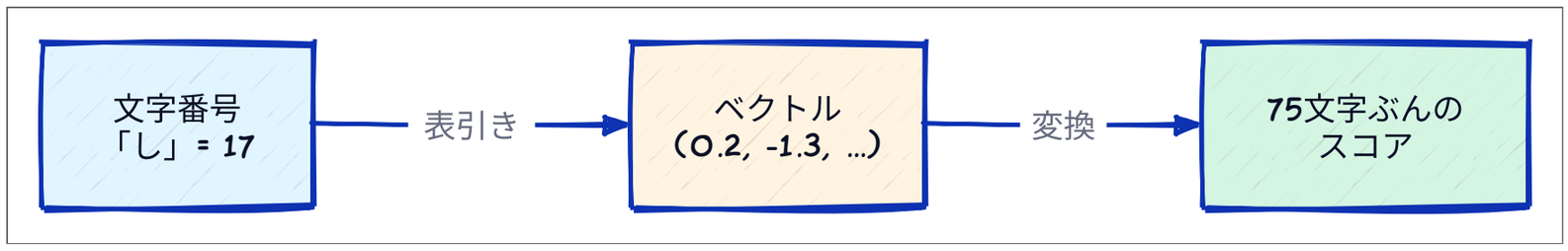
Result

```
tensor([[ 0.,  1.,  2.],
        [ 3.,  4.,  5.],
        [ 6.,  7.,  8.],
        [ 9., 10., 11.]])
tensor([[6., 7., 8.]])
tensor([6., 7., 8.])
```

- **`X @ W`** と **`W[2]`** は**同じもの**。one-hot の掛け算の正体は**表引き**
- つまり前回のモデルは「**文字ごとに1本のベクトル**（表の行）を持っていた」だけ

見方を変えると、世界が変わる

- 前回の W：各文字が「次に来る文字のスコア75個」という行を持つ
- この行を「**その文字の性格を表すベクトル**」と読み替える



- こう分けると、真ん中のベクトルの長さは**75である必要がない**
- 好きな長さ d のベクトル＋あとで75に戻す変換（線形1層）に分解できる
- この「文字→d次元ベクトル」の表を**埋め込み**（**embedding**）と呼ぶ

どこまで縮められる？

> **クイズ**
>
> 『吾輩は猫である』は語彙3,045。バイグラム表は 3,045×3,045 = **927万個**の数。 埋め込みを**2次元**にしたら、パラメータはいくつになる？

- 埋め込み表 3,045×2 ＋ 戻す変換 2×3,045（＋バイアス3,045）＝ **15,225個**
- **609分の1**。そんなに潰して、まともに動くのか？ — 実験で確かめる
- おまけ：$d = 2$ なら、全文字を**1枚の地図**として描ける

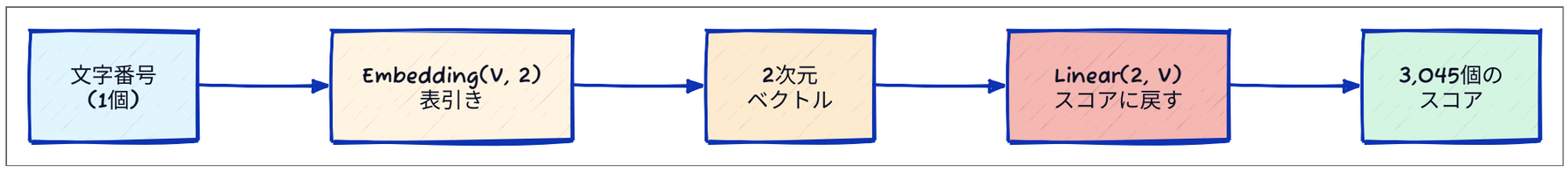
32万組を一口ずつ — ミニバッチ

- 今日の訓練データは『猫』全文の**32万組**。毎歩これ全部で勾配を計算すると、1歩が重すぎる
- そこで毎回、**ランダムに選んだ1,024組だけ**で損失と勾配を計算して1歩進む — **ミニバッチ**（**mini-batch**）



- 一部で測った勾配は「だいたい正しい向き」— **1歩の質を少し落として、歩数を大量に稼ぐ**作戦
- 副作用：毎回違う1,024組で測るので、表示される損失は**小刻みに揺れる**。壊れたのではない

実験：3,045文字を2次元に押し込む



Code

```
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.set_num_threads(1)
text = open("data/neko.txt", encoding="utf-8").read()
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}
V = len(chars)

data = torch.tensor([stoi[c] for c in text])
xs, ys = data[:-1], data[1:]                # （いまの文字, 次の文字）32万組

torch.manual_seed(0)
model = nn.Sequential(nn.Embedding(V, 2),   # 文字 → 2次元ベクトル（表引き）
                      nn.Linear(2, V))      # 2次元 → 3045文字のスコア
opt = torch.optim.AdamW(model.parameters(), lr=2e-2)
g = torch.Generator().manual_seed(0)

for it in range(1, 1001):
    ix = torch.randint(len(xs), (1024,), generator=g)   # ミニバッチ
    opt.zero_grad()
    loss = F.cross_entropy(model(xs[ix]), ys[ix])
    loss.backward()
    opt.step()
    if it in (1, 100, 500, 1000):
        print(f"更新 {it:4d}回 | Loss: {loss.item():.4f}")
```

Result

```
更新    1回 | Loss: 8.2221
更新  100回 | Loss: 5.3674
更新  500回 | Loss: 4.9477
更新 1000回 | Loss: 4.7940
```

- **`nn.Embedding(V, 2)`** が「表引き」の正式な部品。中身はただの V×2 行列
- **`nn.Linear(2, V)`** は「掛けて足す」線形の変換（＋バイアス）、**`nn.Sequential`** は部品を順につなぐ箱
- ミニバッチの実装は **`torch.randint`** の1行 — 引いた番号の組だけで損失を計算する

609分の1に潰した成績は

- 比較対象：**フルサイズのバイグラム表**（数えて作る927万個版）の損失

Code

```
import numpy as np

text = open("data/neko.txt", encoding="utf-8").read()
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}
ids = np.array([stoi[c] for c in text])

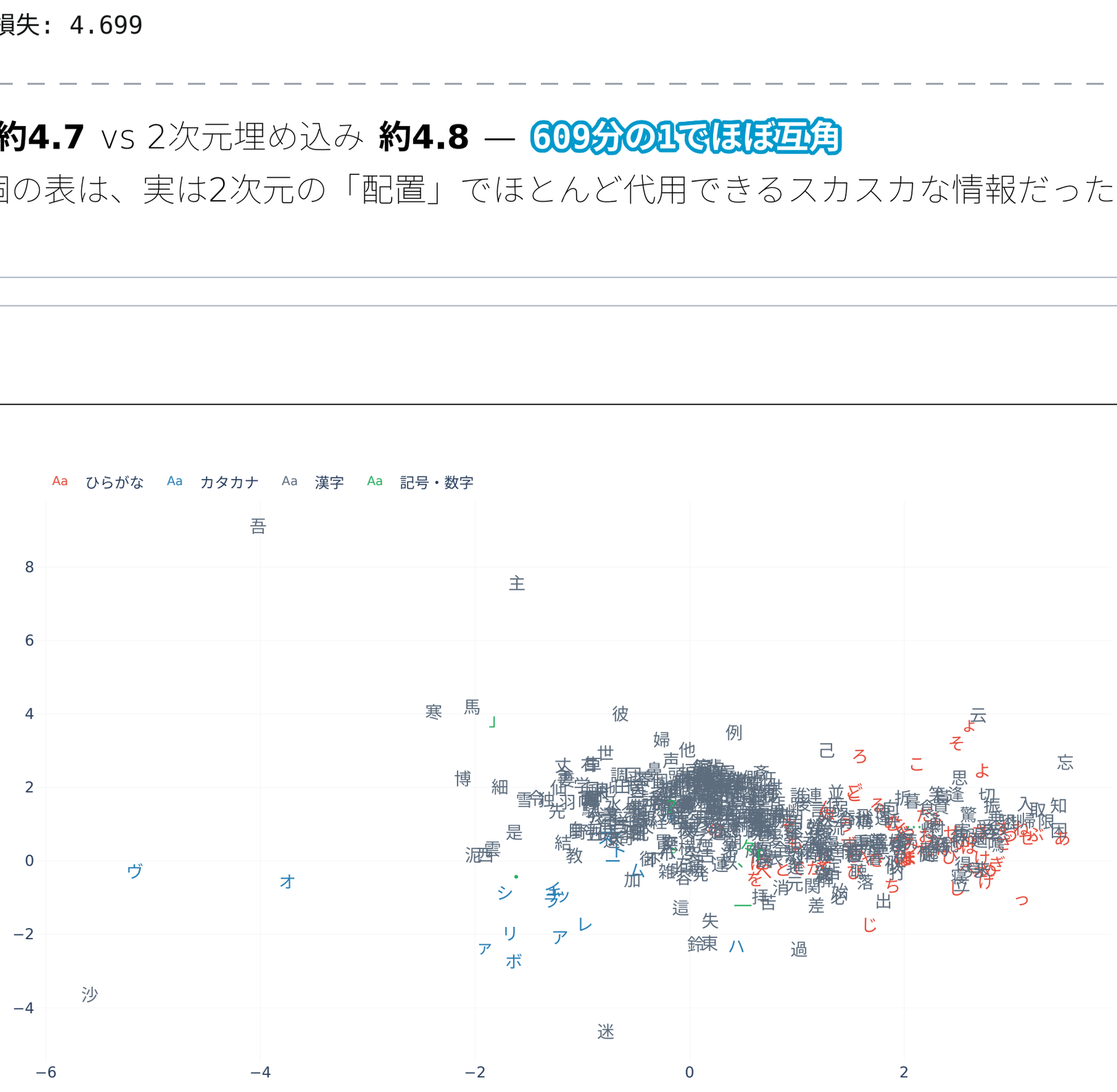
N = np.zeros((len(chars), len(chars)))
np.add.at(N, (ids[:-1], ids[1:]), 1)      # 2次元版「全部数える」
P = (N + 1) / (N + 1).sum(axis=1, keepdims=True)
print("フル表の損失:", round(float(-np.log(P[ids[:-1], ids[1:]]).mean()), 3))
```

Result

```
フル表の損失: 4.699
```

- フル表 **約4.7** vs 2次元埋め込み **約4.8** — **609分の1でほぼ互角**
- 927万個の表は、実は2次元の「配置」でほとんど代用できるスカスカな情報だった

地図を描く



- 『猫』で50回以上出る**525文字**の地図。**ドラッグで範囲ズーム**・ダブルクリックで全体・文字に触れると出現回数。凡例クリックで種類ごとに消せる
- 縦横の軸に意味はない。**近さ**だけが情報
- 全体を眺めると、雲のはるか外に**一匹だけ**離れた文字がいるはず — 覚えておくこと

▸ この地図を自分で描くコード（pip install plotly が必要）

地図から見つかるもの

- **文字の種類ごとに住み分け**が始まっている — ひらがな・カタカナ・漢字・記号。誰も教えていない
- **漢数字が寄り集まる**：一・二・三・四・五・六・七・十がほぼ同じ場所に（「百」「半」も近くに来る）
- **「吾」は一匹狼**：地図のはるか外に1文字だけ

クイズ

なぜ「吾」だけ仲間はずれの場所に置かれた？この小説ならではの理由がある。

- 「吾」の次は**90%「輩」**（535回中481回）— 他のどの文字とも違う極端な性格だから、専用の場所をもらった

意味は「使われ方」に宿る

- モデルは文字の意味も種類も教わっていない。見たのは**どんな文脈で使われるか**だけ
- 使われ方が似た文字は、同じ場所に置くと損失が下げやすい → 自然に寄り集まる
- 「単語の意味はその使われ方が決める」— **分布仮説**（distributional hypothesis）。現代のLLMの土台になっている考え方

ノート

実物のLLMは $d = 2$ ではなく数千次元。地図としては描けないが、「近い＝似た使われ方」の原理は同じ。2次元で見たくなったら第1コースの**PCA**の出番。

部品の対応表

今日の道具	正体
`nn.Embedding(V, d)`	V×d 行列から行を1本引くだけ（前回の W の入口側の役）
`nn.Linear(d, V)`	掛けて足す変換でスコアに戻す（前回の W の出口側の役）
ミニバッチ	毎回ランダムな一部だけで勾配を出して1歩進む

- **新しい概念は「表引きを分解した」ことだけ**。ほかは今日までに開けた部品の組み合わせ

Tweak 地図を揺らしてみる



- **学習のコード** を書き換えて、地図の描画まで実行

1	`torch.manual_seed(0)` → `(1)` など	地図の向きや裏表が変わる。では「吾」の孤立や数字の集まりは？
2	`range(1, 1001)` → `range(1, 301)`	学習途中の「できかけの地図」が見える
3	`nn.Embedding(V, 2)` → `(V, 8)`（描画は最初の2軸）	損失はどうなる？地図は読みやすくなる？

ここで観察

- シードを変えると配置は回転・裏返りするが、**「吾」はどのシードでも外れた場所**に行く。偶然ではなく、データがそうさせている
- レアな文字（数回しか出ない漢字）は初期値からほぼ動かない — 地図の中央の雲はほとんど「未学習」

まとめ

- one-hot × 行列の正体は**表引き** — だから `nn.Embedding` は行列1枚
- 表引きを「d次元ベクトル＋戻す変換」に分解したものが**埋め込み**。609分の1に潰しても互角だった
- 地図では、**使われ方が似た文字が勝手に寄り集まる**（漢数字の群れ、「吾」の孤立）
- 意味 ≒ 使われ方 ≒ 地図上の場所 — 分布仮説

現在の設計図　文字番号 → **埋め込みベクトル** → 線形変換 → 語彙スコア

次回予告

文字がベクトルになった。となれば複数の文字をまとめて入れられる — **窓を広げて**、バイグラムの「記憶1文字」の壁を越えに行く。

練習問題

以下の問題を **Pythonコード** で解け。

問題1：漢数字の群れを確かめる

地図に「一二三四五六七八九十」だけを **目立つ色の別トレース** として重ね、群れから一番外れている漢数字を挙げよ。

0. **neko.txt** を **`data/`** に配置し、本編の学習コード＋地図のコード（折りたたみ）を実行
1. 漢数字10文字の番号を **`stoi`** で取り、**`fig.add_trace(go.Scatter(mode="text", ...))`** で色違いの1本を重ねる
2. ズームして、群れから一番離れている漢数字を目視で特定

問題2：圧縮率の計算

語彙 $V = 3{,}045$ のとき、次の3つのパラメータ数を計算し、フル表に対する圧縮率（何分の1か）をそれぞれ求めよ。

1. フルのバイグラム表（$V \times V$）
2. $d = 2$ の埋め込み分解（$Vd + dV + V$。最後の V はバイアス）
3. $d = 8$ の埋め込み分解

解答例

▸ 解答例を見る

```
Result

    d=2: 15,225 個 — フル表 9,272,025 個の 約609分の1
    d=8: 51,765 個 — フル表 9,272,025 個の 約179分の1
```

- 問題2の含意：d は「文字の性格を書き込む欄の数」。増やすほど表現は豊かに、圧縮は緩くなる — この d が、GPT を組むときに回す主要なツマミ **`n_embd`** の正体